AGROSENSE INDORE

AI Crop Recommendation System for Indore - Malwa Plateau

Simple Presentation & Viva Guide | Easy Version for All Team Members

WHAT IS AGROSENSE INDORE?

AgroSense Indore is a website where a farmer enters soil test values (Nitrogen, Phosphorus, Potassium, pH) and climate values (temperature, humidity, rainfall), and the system uses a trained AI model to tell the farmer which crop is best to grow - along with exactly how much fertilizer to apply per acre. All consultations are saved in MongoDB so the farmer can see their history.

The whole project is split into 5 clear parts - one per team member.

TEAM MEMBERS AND THEIR ROLES

Member	Role	What They Built	Technologies Used
Luv	Frontend & UI Lead	The website interface - sliders, dark mode, bilingual toggle, results display	HTML, CSS, JavaScript
Kunal	Backend & Server Lead	The Express server that receives data from the browser and sends back answers	Node.js, Express.js
Naman	Database & ML Integration Lead	Connects Node to Python AI model via IPC bridge. Saves all data in MongoDB. Calculates fertilizer doses.	MongoDB, Mongoose, Node.js
Arhaan	Documentation & Architecture Lead	Wrote the SRS, designed system diagrams, documented soil science, made data dictionary	SRS, UML, ICAR-IISR Norms
Kuldeep	ML Training & Data Lead	Downloaded NASA climate data, cleaned it, built the dataset, trained 5 AI models, picked the best one (Random Forest 99%)	Python, NumPy, Pandas, Scikit-Learn

HOW TO USE THIS GUIDE:

- Each team member has their own section (2 pages each) with a 1-minute speech script and likely examiner questions + answers.
- Section 1 explains where data came from (NASA, ICAR etc.) - ALL members should read this.
- Section 7 has tricky questions the examiner may ask the whole group together.

SECTION 1: WHERE DID OUR DATA COME FROM?

How the dataset was built using NASA + ICAR standards

IMPORTANT - WHY NOT KAGGLE DATASETS?

Generic crop datasets on Kaggle have crops like Coconut and Apple - which cannot grow in Indore! Indore has deep black soil (Vertisol) and a specific climate. We built our own dataset using REAL climate data from NASA and REAL soil norms from the Indian government.

External Sources Used

Source	Website	What We Got From It
NASA POWER API	power.larc.nasa.gov (Free, no login needed)	5 years (2019-2023) of daily Temperature, Humidity and Rainfall data for all 5 tehsils of Indore district. Kuldeep wrote a Python script to download this automatically.
Soil Health Card Portal (Govt. of India)	soilhealth.dac.gov.in	Real soil profile of Indore's black soil: Nitrogen 20-100, Phosphorus 10-40 kg/ha, Potassium 150-450 kg/ha (naturally high!), pH 7.2-8.2 (alkaline).
ICAR-IISR Indore (Soybean Research)	iisrindore.icar.gov.in Khandwa Road, Indore	Agronomic targets for Soybean (the main Kharif crop of Malwa): NPK target 25:60:40 kg/ha. Certified varieties: JS-9560, JS-2034.
RVSKVV - College of Agriculture, Indore	rvskvv.net	Fertilizer targets for Rabi crops: Wheat (100:50:40), Gram/Chickpea (20:50:30), Maize, Onion, Potato.

How the NASA data was downloaded and cleaned (Kuldeep's work)

Kuldeep wrote ml/build_dataset.py which downloaded real climate data from NASA for the exact GPS coordinates of all 5 tehsils:

Tehsil	GPS Coordinates	5-Year Avg Monsoon Rain	Soil Type
Sanwer	22.97 N, 75.82 E	890 mm	Deep Heavy Black Cotton Soil
Indore	22.71 N, 75.85 E	920 mm	Medium Deep Black Vertisol
Depalpur	22.84 N, 75.54 E	880 mm	Heavy Clay Vertisol
Mhow	22.55 N, 75.76 E	960 mm	Loamy Black Transitional Soil
Hatod	22.79 N, 75.72 E	870 mm	Shallow to Medium Black Soil

The script filtered out NASA's -999.0 missing value flags. Then it split daily data into seasons: Kharif (June-Oct, monsoon crops) and Rabi (Nov-Mar, winter crops). Finally it calculated 5-year seasonal averages for each tehsil.

How the CSV training files were created (build_dataset.py)

Step 1: Pick a random tehsil and season. Load NASA's real climate baseline for that combination.

Step 2: Add small random variation to temp/humidity/rainfall to make it realistic (not all years are identical).

Step 3: Randomly generate soil N, P, K, pH values within the Indore black soil range - INDEPENDENTLY of which crop will be assigned.

Step 4: Apply real agronomic rules to see which crops CAN grow in that soil/climate combination.

Step 5: Assign one of the valid crops and add the row to the dataset.

Step 6: Keep repeating until we have exactly 250 rows for each of 6 crops = 1500 total rows.

File	Size	What It Contains
data/raw/indore_crop_dataset.csv	1500 rows, 11 cols	The main training file: district, tehsil, season, N, P, K, temperature, humidity, ph, rainfall, crop
data/climate_by_tehsil_season.csv	15 rows, 5 cols	NASA 5-year averages per tehsil per season: tehsil, season, temperature_avg, humidity_avg, rainfall_total
data/soil_profile.csv	4 rows, 4 cols	Indore black soil min/max ranges for N, P, K, pH (from Soil Health Card data)
data/data_dictionary.csv	11 rows, 5 cols	Explains every column: what it means, unit, valid range, data source (Arhaan's work)

SECTION 2: LUV - FRONTEND & UI ENGINEERING (Page 1/2)

HTML5, CSS3 Design System, JavaScript, Dark Mode, Bilingual Toggle

LUV'S 1-MINUTE OPENING SPEECH:

"I am Luv, and I built the frontend - which is the website interface that farmers see and interact with. My job was to make it clean, simple, and usable even on a budget mobile phone with slow internet. I used HTML5 for structure, a custom CSS design system for styling, and JavaScript for all the interactive behaviour. Key things I built: (1) True Black Dark Mode with zero white-flash on page reload; (2) Instant English to Hindi language switching; (3) Reactive soil sliders that auto-fill when a tehsil is selected; (4) A history drawer that shows past farmer consultations."

What Each Frontend File Does

File	Who reads it	What it does
frontend/index.html	Browser	The main webpage. Contains all HTML elements: sliders, buttons, result display area, history drawer. Has a tiny inline script in that sets dark/light mode before any CSS loads (prevents white flash).
frontend/style.css	Browser	All visual design. Uses CSS variables (design tokens) so dark and light mode are just a switch. Styles the sliders, cards, confidence bar, fertilizer grid, history drawer, etc.
frontend/app.js	Browser	All interactivity: slider value display, tehsil auto-preset, sending data to the server with fetch(), displaying results, language toggle, dark mode toggle, history loading.

Key Technical Concepts (Simple Explanation)

Dark Mode Zero-Flicker: Normally when you switch to dark mode and refresh the page, you see a white flash for a split second because the browser paints the page before the stylesheet loads. Luv fixed this by putting a tiny 2-line script inside the HTML tag (before CSS). It reads the saved preference from localStorage and sets dark mode immediately. No white flash.

CSS Design Tokens (Variables): Instead of repeating the same color (#065f46) in 50 places, Luv defined it once as --primary: #065f46. To switch dark/light mode, only the values of these variables change. All styles automatically update. This is why dark mode works so cleanly.

Bilingual (Hindi/English): Every text element in the HTML has a data-i18n="key" attribute. The JavaScript app.js has a dictionary of English and Hindi versions of each key. When you click the language button, it loops through all elements and swaps the text. No page reload needed.

Tehsil Auto-Presets: The TEHSIL_PRESETS object in app.js stores NASA's 5-year climate averages for each tehsil. When the farmer selects 'Sanwer', the sliders automatically animate to the correct temperature, humidity and rainfall values for that tehsil.

Fetch API (sending data to server): When the farmer clicks the button, app.js collects all slider values and sends them to Kunal's Express server at the URL /api/recommend using the browser's built-in fetch() function. It then waits for the JSON response and renders the result on screen.

SECTION 2: LUV - FRONTEND & UI ENGINEERING (Page 2/2)

Likely Viva Voce Questions & Answers

Q: Why did you use plain HTML/CSS/JS instead of React or Next.js?

A: Our users are farmers in rural Indore who often have budget Android phones and slow 2G internet. A React app adds 300+ KB of JavaScript that needs to download and execute before anything shows on screen. Our plain HTML/JS approach loads in under 150ms. We still wrote the JavaScript in a modular, component-like style - it just does not need the React library to do so.

Q: How does dark mode work without a white flash on reload?

A: In the HTML , before any CSS link tag, we have a tiny script: const t = localStorage.getItem('agrosense_theme') || 'dark'; document.documentElement.setAttribute('data-theme', t). This runs synchronously before the browser paints anything. So the very first frame is already in dark mode.

Q: How does the language toggle work without reloading the page?

A: Every text element has a data-i18n='key' attribute. The app.js file has a TRANSLATIONS dictionary with English and Hindi versions for each key. The toggleLanguage() function loops through all elements with that attribute and swaps the text. Saved to localStorage so preference persists across visits.

Q: How do the sliders auto-fill when a tehsil is selected?

A: There is a TEHSIL_PRESETS object in app.js that stores the NASA 5-year climate averages for each tehsil. When onchange fires on the select dropdown, the applyTehsilPreset() function reads the values from this object and sets the slider values programmatically.

Q: How does the confidence bar animation work?

A: The confidence bar is a CSS div with width set via JavaScript: confidenceFill.style.width = confidence + '%'. A CSS transition: width 0.6s ease makes it animate smoothly to the correct width.

LUV'S DEMO CHECKLIST:

1. Toggle dark/light mode and refresh the page - no white flash should appear.
2. Switch between English and Hindi - text changes instantly without reload.
3. Select 'Sanwer' from the tehsil dropdown - sliders animate to preset values.
4. Click the history button (bottom right) - shows past farmer consultations.

SECTION 3: KUNAL - BACKEND & SERVER ENGINEERING (Page 1/2)

Node.js, Express.js REST API, CORS, Request Validation, Server Architecture

KUNAL'S 1-MINUTE OPENING SPEECH:

"I am Kunal, and I built the backend server. The backend is the bridge between the farmer's browser and the AI model and database. I used Node.js with Express.js to build a REST API. When the farmer clicks the button, the browser sends a POST request to my server at /api/recommend. My server receives the soil data, validates it, asks Naman's IPC bridge to get the AI prediction, saves the result to MongoDB, and sends the answer back to Luv's UI. I also configured CORS security, input validation, and environment variables."

API Routes - The Server's Menu

Think of routes as the menu of a restaurant. The browser is the customer, and the server is the kitchen:

HTTP Method + Route	Who calls it	What happens
POST /api/recommend	Browser (Luv's app.js)	Receives soil + farmer data. Validates it. Asks Python AI for prediction. Calculates fertilizer. Saves to MongoDB. Returns recommendation JSON.
GET /api/history	Browser (history drawer)	Fetches the last 20 recommendation records from MongoDB, sorted newest first, and returns them as JSON.
GET /health	Monitoring / DevOps	Returns {status: 'running'} to confirm the server is alive. Useful for uptime monitoring.

How server.js Starts Up

1. Loads .env file (dotenv.config()) to read PORT number and MongoDB URI without hardcoding secrets in code.
2. Creates Express app and attaches middleware: cors() allows browser requests, express.json() reads JSON data from requests.
3. Registers our API routes under /api/ prefix.
4. Calls db.connectDB() to connect to MongoDB Atlas.
5. Starts listening on PORT 5001 for incoming HTTP requests from browsers.
6. Serves the frontend HTML/CSS/JS files so the user does not need a separate web server.

What Happens Inside POST /api/recommend

This is the most important route. Step by step:

1. Extract values from request body: N, P, K, ph, rainfall, temperature, humidity, tehsil, farmer_name, village.
2. Validate: if N, P, K or ph are missing, immediately return HTTP 400 error with a clear message.
3. Convert all values to numbers with parseFloat() - they come as strings from HTML forms.
4. Call mlBridge.predictCrop(soilData) - this is Naman's file that runs Python. Wait for the result.
5. Call db.saveRecommendation() - save the full consultation to MongoDB.
6. Send back JSON with the recommendations and fertilizer plan.

SECTION 3: KUNAL - BACKEND & SERVER ENGINEERING (Page 2/2)

Likely Viva Voce Questions & Answers

Q: Why did you choose Node.js/Express instead of Flask or Django (Python)?

A: Node.js uses an event-driven, non-blocking I/O model. This means it can handle many simultaneous browser connections without creating a new thread for each one. For an API gateway that mainly does I/O (receive request, call Python, save to DB, send response), Node.js is very efficient. The heavy CPU computation (the AI prediction) is handled separately by Python. We get the best of both worlds.

Q: What is CORS and why do you need it?

A: CORS stands for Cross-Origin Resource Sharing. Browsers block AJAX requests to a different domain or port by default as a security measure. During development, the frontend might run on port 3000 while our server is on port 5001. Without CORS middleware, the browser would block our API calls. We added cors() middleware which adds the right HTTP headers to allow browser requests.

Q: How do you protect against bad input data?

A: In api.js, we first check that N, P, K and pH are present. Then we use parseFloat() on all values - if someone sends a text string like 'abc', parseFloat returns NaN. We check for NaN values and return a 400 Bad Request with a friendly error message. This prevents invalid data reaching the AI model or database.

Q: What are environment variables and why use them?

A: Environment variables (in the .env file) store configuration like database passwords and port numbers separately from the code. If we put the MongoDB password directly in server.js and committed it to GitHub, anyone could steal it. With dotenv, the .env file stays on our machine only and is listed in .gitignore. In code we use process.env.MONGODB_URI to access it safely.

Q: If the server crashes, what happens to ongoing requests?

A: We use try-catch blocks in all our async route handlers. If any step throws an error, the catch block runs and sends HTTP 500 (Internal Server Error) with a safe error message. The Node.js process itself does not crash because Express catches unhandled errors through global error middleware.

SECTION 4: NAMAN - DATABASE & ML INTEGRATION (Page 1/2)

MongoDB, Mongoose Schemas, Node-to-Python IPC Bridge, NPK Fertilizer Math

NAMAN'S 1-MINUTE OPENING SPEECH:

"I am Naman, and I handled two things: the database and the connection to the AI model. For the database, I used MongoDB with Mongoose. I designed the data schema for saving farmer consultations - including the soil inputs and fertilizer plan embedded directly in one document for fast saves and reads. For the AI connection, I built an IPC bridge in ml_bridge.js that launches Python as a subprocess, sends it the soil data as JSON over stdin, waits up to 5 seconds for the answer, and returns it to Kunal's route handler. I also wrote the fertilizer math - calculating how many kg of Urea, DAP and MOP the farmer needs based on the recommended crop's nutrient targets from ICAR-IISR."

MongoDB Schema Design (Recommendation.js)

We use MongoDB (a document database). Every farmer consultation is saved as one document. The document has sub-documents embedded inside it:

Field / Sub-Schema	Type	What it stores
farmer_name	String	Name the farmer typed in (optional, defaults to 'Farmer Guest')
tehsil	String	Which of Indore's 5 tehsils the farmer is from
soil_inputs (embedded)	Object	All 7 soil/climate values: N, P, K, ph, temperature, humidity, rainfall
primary_crop	String	The #1 recommended crop (e.g. 'Soybean')
confidence	Number	How confident the AI is (e.g. 94.2 means 94.2%)
alternative_crops	Array	2nd and 3rd best crop suggestions
fertilizer_plan (embedded)	Object	Urea kg, DAP kg, MOP kg, organic compost, and application schedule note
created_at	Date	Timestamp of when this consultation was saved (auto-set by MongoDB)

Why Embed Sub-Documents Instead of Separate Collections?

In SQL databases you would have separate tables and JOIN them. In MongoDB, we embed the SoilInput and FertilizerPlan directly inside the Recommendation document. Why?

- A farmer consultation is one complete event - the soil, the prediction and the fertilizer plan all belong together.
- Embedded = one single write to MongoDB saves everything atomically. No risk of partial saves.
- When reading history, one query returns the full consultation - no JOIN needed. Much faster.
- Agronomic consultations are immutable records - we never need to update just the soil part independently.

The IPC Bridge (ml_bridge.js) - How Node.js Talks to Python

Node.js and Python are separate programs. They cannot call each other's functions directly. Naman built a bridge using Child Process IPC (Inter-Process Communication):

1. Naman's ml_bridge.js uses Node's built-in child_process.spawn() to launch python3 and run predictor_service.py.
2. The soil data (7 numbers) is serialized as a JSON string and passed as a command-line argument to Python.
3. Python reads the argument, runs the AI model, and prints the result as a JSON string.
4. Node.js reads Python's stdout output, parses the JSON, and returns it to api.js.
5. If Python takes more than 5 seconds OR crashes, a safety timer kicks in and returns a fallback answer so the farmer always gets a response.

SECTION 4: NAMAN - DATABASE & ML INTEGRATION (Page 2/2)

Fertilizer Math & Likely Viva Voce Questions

Fertilizer Calculation Math (Simple Version)

After the AI picks the best crop, we need to tell the farmer how much fertilizer to buy. The math is based on ICAR-IISR targets for each crop:

CROP NUTRIENT TARGETS (from ICAR-IISR research):

Soybean: N=25, P=60, K=40 kg/ha | Wheat: N=100, P=50, K=40

Gram/Chickpea: N=20, P=50, K=30 | Maize: N=85, P=55, K=40

Onion: N=90, P=50, K=80 | Potato: N=110, P=70, K=90

Step by step fertilizer calculation:

1. How much does this soil NEED? Deficit_N = max(0, Target_N - Soil_N). Same for P and K.
2. DAP (18% Nitrogen, 46% Phosphorus): Use DAP to cover the phosphorus deficit. DAP_kg = Deficit_P / 0.46. DAP also brings some nitrogen: N_from_DAP = DAP_kg x 0.18.
3. Urea (46% Nitrogen): Cover the remaining nitrogen deficit. Urea_kg = (Deficit_N - N_from_DAP) / 0.46. We deduct N_from_DAP so we don't over-apply nitrogen.
4. MOP (60% Potassium): Cover the potassium deficit. MOP_kg = Deficit_K / 0.60.
5. Apply schedule: 100% DAP + 100% MOP at sowing. Urea in 2 parts: 50% at sowing, 50% at 25-30 days. (Splitting Urea prevents wastage in Indore's black soil.)

Naman's Viva Questions & Answers

Q: Why did you use MongoDB instead of MySQL?

A: Our project spec says MongoDB. MongoDB is a document database - perfect for storing farmer consultations which are complex nested objects (soil readings inside the consultation, fertilizer plan inside the consultation). MongoDB lets us save this whole consultation as one atomic write. MySQL would require 3 separate tables and JOIN queries.

Q: What does 'embedded subdocuments' mean in MongoDB?

A: In Mongoose (MongoDB ORM), instead of linking to a separate collection via a foreign key (like SQL), we define SoilInputSchema and FertilizerPlanSchema and embed them directly inside the parent RecommendationSchema. When we save a Recommendation, all three pieces save together in one document, in one database operation.

Q: What happens if Python crashes while running the AI model?

A: ml_bridge.js has a 5-second watchdog timer (setTimeout). If Python does not respond in 5 seconds, we kill the Python process and return a safe fallback answer based on ICAR rules. This way the farmer still gets a recommendation and the server never freezes. We also listen to the 'error' event from spawn() in case Python cannot start at all.

Q: Why do you subtract N_from_DAP before calculating Urea?

A: DAP (18:46:0) provides both Phosphorus AND Nitrogen. If we calculate Urea based on the full nitrogen deficit without accounting for what DAP already provides, we would over-apply nitrogen. Excess nitrogen in Indore's black soil causes plants to grow too many leaves, delayed grain filling, and pest attacks. The deduction ensures precision.

Q: How do you ensure the database index is efficient for the history query?

A: We sort consultations by created_at in descending order. Mongoose/MongoDB uses a B-tree index on this field. Querying the last 20 records on an indexed date field takes under 5ms regardless of how many total records exist in the collection.

SECTION 5: ARHAAN - DOCUMENTATION & ARCHITECTURE (Page 1/2)

SRS, UML Diagrams, Black Soil Science, Data Dictionary

ARHAAN'S 1-MINUTE OPENING SPEECH:

"I am Arhaan, and I handled the documentation and system architecture. I wrote the Software Requirements Specification (SRS) which defines what the system must do and how well it must do it. I built UML diagrams showing how all the parts connect. I researched the soil science of Indore's black soil (Vertisols) and aligned our software rules with ICAR-IISR and RVSKVV agricultural standards. I also created the data dictionary that explains every column in our dataset."

Software Requirements Specification (SRS)

The SRS is a formal document that describes what the system must do. It has two types of requirements:

Requirement	Type	Description
FR-1: Crop Prediction	Functional	Given 7 soil/climate inputs, predict the best crop with confidence score in under 500ms.
FR-2: Fertilizer Calculation	Functional	Calculate exact Urea, DAP, MOP doses per acre with split application schedule.
FR-3: Tehsil Presets	Functional	Selecting a tehsil auto-fills climate sliders with real NASA 5-year averages.
FR-4: Bilingual UI	Functional	Instant English-Hindi toggle with no page reload.
NFR-1: Speed	Non-Functional	Full page load under 150ms. API response under 250ms.
NFR-2: Reliability	Non-Functional	If Python AI crashes, fallback rules give a response. Zero server downtime.
NFR-3: Accessibility	Non-Functional	WCAG 2.1 AA: minimum 7:1 contrast, 48px touch targets, color not used alone for info.

Indore Black Soil Science (Vertisols) - Why It Matters

The whole project is designed around Indore's specific soil. Understanding it is important:

Vertisols (Black Cotton Soil): Indore district has deep black soil rich in montmorillonite clay (45-60%). It swells when wet (monsoon), shrinks and cracks when dry (winter). This is why split fertilizer application is needed.

High Native Potassium: Indore black soil naturally has a lot of Potassium (150-450 kg/ha). So MOP fertilizer is only needed for crops that are heavy potassium consumers (Onion, Potato).

Low Nitrogen: The soil is low in available nitrogen because heat oxidizes organic matter. Crops need external nitrogen from Urea or DAP.

Alkaline pH (7.2-8.2): The soil is slightly alkaline. Crops that need acidic soil (like Coffee, Tea, Blueberry - pH 4-5) cannot grow here. Our AI model enforces this pH constraint.

Soybean is the Kharif King: ICAR-IISR research in Indore shows Soybean (varieties JS-9560, JS-2034) is the best Kharif crop. It fixes its own nitrogen through root nodules, perfect for low-N black soil.

SECTION 5: ARHAAN - DOCUMENTATION & ARCHITECTURE (Page 2/2)

Viva Voce Questions & Answers

Q: What is the purpose of an SRS in a software project?

A: The SRS (Software Requirements Specification) is the contract between the client (farmers, agricultural officers) and the development team. It specifies exactly what the system must do (functional requirements) and how well it must do it (non-functional: speed, reliability, accessibility). Without an SRS, different team members might build incompatible parts. Our SRS made sure Luv's UI, Kunal's API, Naman's database, and Kuldeep's AI model all agreed on the same input/output format.

Q: Why is Indore's soil called Vertisol and why does it matter for this project?

A: Vertisols are soils dominated by swelling clay minerals (montmorillonite). They expand when wet and crack when dry. This physical property means nitrogen fertilizer leaches away easily if applied all at once, which is why we prescribe a split application (50% at sowing, 50% at 25-30 days). The high native potassium means we do not need to prescribe MOP for every crop. These soil-specific agronomic decisions make AgroSense Indore accurate for this region specifically.

Q: Why did you create a data dictionary and what does it contain?

A: A data dictionary is a formal reference document explaining every piece of data in the system. Our data_dictionary.csv has 11 rows (one per feature). For each: the column name, data type, valid range, unit of measurement, which source the data came from (NASA POWER vs ICAR vs Soil Health Card), and the agronomic meaning. Without this, another researcher or software engineer could misinterpret, say, 'K' as Kelvin temperature instead of Potassium.

Q: What UML diagrams did you prepare for this project?

A: I prepared: (1) Component Diagram showing how the 5 modules (Frontend, Express Server, IPC Bridge, Python Predictor, MongoDB) connect; (2) Sequence Diagram tracing one full farmer recommendation request from browser click to response; (3) Entity-Relationship diagram for the MongoDB schema; (4) Activity Diagram for the fertilizer calculation decision flow.

Q: What are the social benefits of AgroSense Indore for farmers?

A: Three main benefits: (1) Economic - precise fertilizer dosing saves farmers Rs. 2000-4500 per acre by preventing over-purchasing of chemicals; (2) Ecological - prevents nitrogen and phosphorus runoff into the Kshipra and Gambhir rivers which cause algae bloom; (3) Social - the bilingual English-Hindi interface empowers farmers who cannot read English.

SECTION 6: KULDEEP - ML TRAINING & DATA CLEANSING (Page 1/2)

NumPy, Pandas, Scikit-Learn, 5 Models Compared, Random Forest Wins

KULDEEP'S 1-MINUTE OPENING SPEECH:

"I am Kuldeep, and I handled the data engineering and machine learning. First I wrote a Python script that downloaded 5 years of daily climate data from the NASA POWER API for all 5 tehsils of Indore. I cleaned the data by removing -999 missing value flags and calculated seasonal averages for Kharif and Rabi. Then I helped build the dataset CSV with 1500 balanced rows. Next I trained 5 different AI classification algorithms and compared them using 5-Fold Cross Validation. The winner was Random Forest with 99% accuracy. I saved the trained model as a .pkl file which Naman's predictor_service.py loads to make predictions."

The 7 Features (Inputs) the AI Learns From

Feature	Unit	Range in Dataset	Why It Matters for Crop Prediction
N (Nitrogen)	index	20 - 100	Low N -> good for legumes (Soybean, Gram). High N -> good for cereals (Wheat, Maize).
P (Phosphorus)	kg/ha	10 - 40	High P needed for Potato and Onion. Important for root development.
K (Potassium)	kg/ha	150 - 450	High K separates tuber crops (Potato, Onion) from others.
Temperature	deg C	15 - 35	Kharif crops need high temp (28-35C). Rabi crops need cool (15-22C). Very decisive feature.
Humidity	%	25 - 95	Monsoon Kharif crops need high humidity. Dry Rabi crops prefer low humidity.
pH	pH	6.5 - 8.2	Enforces which crops can survive in Indore's alkaline Vertisol.
Rainfall	mm	15 - 1250	MOST important. Kharif = 800-1200mm. Rabi = 20-60mm. Clear separator.

5 AI Models Compared Using 5-Fold Cross Validation

Kuldeep tested 5 different algorithms. For each, he used 5-Fold Stratified Cross Validation - split the 1500 samples into 5 equal groups, train on 4, test on 1, repeat 5 times. Average score = reliable accuracy:

Algorithm	5-Fold CV Accuracy	Test Accuracy	Speed	Verdict
Random Forest	98.8% +/- 0.5%	99.0%	< 25ms	[WINNER] Best overall
Extra Trees	98.4% +/- 0.6%	98.6%	< 20ms	Close second
Gradient Boost	97.5% +/- 0.8%	97.8%	~95ms	Too slow
Decision Tree	93.8% +/- 1.2%	94.2%	< 5ms	Overfits, unstable
Logistic Reg.	85.9% +/- 1.5%	86.4%	< 5ms	Cannot learn curves

SECTION 6: KULDEEP - ML TRAINING & DATA CLEANSING (Page 2/2)

Why Random Forest Won & Likely Viva Questions

Why Random Forest Beats a Single Decision Tree

A Decision Tree gets 94.2% but Random Forest gets 99.0%. Why?

Single Decision Tree: Tries to find perfect rules to separate all 1500 training samples. Goes very deep and 'memorizes' the training data. Works great on training data but makes mistakes on new data (overfitting).

Random Forest = 200 Trees voting: Instead of one deep tree, we build 200 separate decision trees. Each tree is trained on a random sample of rows AND a random subset of features. So each tree is different and sees the data from a different angle.

Voting reduces errors: If tree #1 makes a wrong prediction for a sample, trees #2 through #200 might get it right. The final answer is by majority vote across all 200 trees. Random individual errors cancel out.

Result: The ensemble is more stable (low variance), less prone to memorizing noise, and generalizes better to new farmers' soil data.

Most Important Features (Gini Importance)

After training, Random Forest tells us which features were most useful for making decisions. For Indore crops:

1. Rainfall (highest importance) - completely separates Kharif (800-1200mm) from Rabi (20-60mm) crops.
2. Temperature - secondary seasonal separator (28-35C for Kharif vs 15-22C for Rabi).
3. Nitrogen - separates legumes (low N: Soybean, Gram) from cereals (high N: Wheat, Maize).
4. Phosphorus and Potassium - separates tuber crops (Potato, Onion need high P and K) from others.
5. pH and Humidity - boundary constraints (pH enforces Vertisol alkalinity limits).

Kuldeep's Viva Questions & Answers

Q: Why did you choose Random Forest over a Neural Network?

A: For our 1500-row tabular dataset, Random Forest consistently outperforms neural networks. Neural networks need large amounts of data (tens of thousands of rows) to learn reliably. On small datasets, they overfit easily. Random Forest also needs no feature scaling (no need to normalize N, P, K values), trains in under 1 second, and provides feature importance which helps us explain to farmers WHY a crop was recommended.

Q: What is 5-Fold Stratified Cross Validation and why use it?

A: We split our 1500 samples into 5 equal groups (folds). 'Stratified' means each fold has the same proportion of all 6 crops (250 each total, so 50 per crop per fold). We train on 4 folds, test on 1, repeat 5 times with a different test fold each time, and average the accuracy. This gives a more reliable accuracy estimate than a single 80-20 train-test split, which could get lucky or unlucky with one particular split.

Q: What does the model.pkl file contain?

A: We use joblib.dump() to save a Python dictionary containing: the trained RandomForestClassifier object, the list of feature names in the exact order used for training, the list of 6 crop class names, the accuracy score, and metadata about the district and zone. The predictor_service.py loads this dictionary to ensure it feeds features to the model in exactly the same order as during training.

Q: Why is 'Rainfall' the most important feature by Gini importance?

A: Rainfall perfectly separates the two seasons. Kharif crops (Soybean, Maize) grow in 800-1200mm monsoon rainfall. Rabi crops (Wheat, Gram, Onion, Potato) grow with only 20-60mm of irrigation. No Kharif crop overlaps with Rabi rainfall ranges. So at the very first tree node, checking rainfall correctly classifies about 60% of samples. Features that resolve uncertainty in early tree nodes get higher Gini importance.

Q: How did you handle the imbalanced nature of crop representation in real life?

A: In real Indore fields, Soybean might cover 60% of land but Potato only 5%. If we sampled naturally, the model would always predict Soybean and still score 60% accuracy. We deliberately created a BALANCED dataset with exactly 250 samples per crop using quota-based sampling in build_dataset.py. This forces the AI to learn the differences between all 6 crops equally.

SECTION 7: GROUP VIVA DEFENSE STRATEGY

Tough cross-team questions the examiner may ask + team answer strategy

GROUND RULES FOR GROUP VIVA:

- When examiner asks about UI or dark mode -> Luv answers.
- When examiner asks about server, routes, CORS -> Kunal answers.
- When examiner asks about database, IPC bridge, fertilizer math -> Naman answers.
- When examiner asks about SRS, soil science, data dictionary -> Arhaan answers.
- When examiner asks about data cleaning, model training, accuracy -> Kuldeep answers.
- For end-to-end questions, each person answers their own part in sequence.

End-to-End Flow Question (Everyone Answers in Turn)

Examiner: Walk me through exactly what happens when a farmer fills in the form and clicks the button.

Luv: The farmer's browser collects all 7 slider values and farmer name. app.js calls fetch('/api/recommend', {method: 'POST', body: JSON}). A loading spinner appears.

Kunal: Express receives the POST request at /api/recommend in api.js. It validates N, P, K, pH are present. Converts to numbers. Calls mlBridge.predictCrop(soilData).

Naman (IPC): ml_bridge.js spawns Python as a subprocess. Sends soil data as JSON string. Starts 5-second watchdog timer. Waits for stdout output from Python.

Kuldeep: predictor_service.py reads the JSON. Loads indore_crop_model.pkl. Builds a 7-element feature vector [N,P,K,temp,humidity,pH,rainfall]. Runs model.predict_proba(). Sorts by confidence. Prints top-3 crops as JSON.

Naman (Fertilizer + DB): ml_bridge.js reads Python's output. Back in api.js, we calculate Urea/DAP/MOP doses. Call db.saveRecommendation() to save to MongoDB.

Luv: Express sends back JSON. app.js receives it. Updates the crop name, confidence bar, alternative crops, and fertilizer grid. History drawer updates.

Other Tough Cross-Cutting Questions

Q: Why is your system better than just telling farmers to always grow Soybean in Kharif?

Team Answer (Kuldeep + Naman + Arhaan): Simple rules ('always grow Soybean') ignore soil chemistry. A field with pH 9.0 or very low phosphorus is not suitable for Soybean. Our model considers all 7 features simultaneously. It also gives alternative options when multiple crops are viable, plus exact fertilizer doses per acre - something a simple rule cannot do.

Q: Why can't this system be used for Bhopal or Ujjain farmers?

Team Answer (Kuldeep + Arhaan): Our AI was trained only on data from Indore's 5 tehsils using NASA coordinates specific to Indore district and Indore's Vertisol (black soil) profile. Bhopal has different soil types and different climate. Ujjain has a different rainfall pattern. Using our model there would give wrong predictions. The fix is straightforward: download NASA data for those districts' GPS coordinates, get their Soil Health Card data, retrain the model, and deploy a new version.

Q: Your model has 99% accuracy - isn't that suspiciously too high?

Team Answer (Kuldeep): This is a common and fair question! The high accuracy is because our 6 crops naturally separate very clearly in the feature space. Soybean only grows in Kharif (high rainfall, high temp). Wheat only grows in Rabi (low rainfall, cool temp). These are almost perfectly orthogonal. The trickier cases are within Rabi (Wheat vs Gram vs Onion vs Potato) where N, P, K separate them. The 5-Fold CV standard deviation of 0.5% confirms it is stable and not just a lucky split.

INDIVIDUAL DEMO CHECKLIST BEFORE VIVA:

[Luv] Show dark mode, Hindi toggle, tehsil preset auto-fill, and history drawer.

[Kunal] Show server.js, api.js routes, and a live Postman/cURL call to /api/recommend.

[Naman] Explain ml_bridge.js watchdog timer and write the fertilizer DAP formula on board.

[Arhaan] Walk through the SRS table and explain why Soybean suits Indore black soil.

[Kuldeep] Show the 5-model benchmark table and explain why Random Forest won over Decision Tree.